

Project 3

zkVM-guest API Standardization

7th Breakout Call · standards recap, a new proposal, and a handover

github.com/eth-act/zkvm-standards

Standards Recap

Accepted (navy)

In review (amber)

10 ACCEPTED

1 IN REVIEW

4 ON HOLD — slides 5 to 8

1 ACCEPTED

RISC-V Target

Minimal ISA riscv64im_zicclsm — one 64-bit target for every zkVM.

2 ACCEPTED

IO Interface

Portable read_input / write_output for private input, public output.

3 ACCEPTED

Cryptographic Accelerators

A C interface for the EVM precompile operations.

4 ACCEPTED

Termination Semantics

Success may produce a proof; failure must not.

5 ACCEPTED

Memory Layout Restrictions

Each vendor owns its memory map; portability stays at source level.

6 ACCEPTED

Memory Safety Guard Regions

Null-pointer trap and stack guard — violations abort.

7 ACCEPTED

Instruction Address Misaligned

A misaligned jump aborts. No rounding, no recovery.

8 ACCEPTED

Static Library & Linker Script

The vendor-supplied guest runtime.

9 ACCEPTED · NEW · #33

ELF Loading & Validation

The ELF is the authoritative initial machine state.

10 ACCEPTED · NEW · #32

Accelerated Memory Operations

Vendor memcpy / memmove / memset / memcpy.

11 IN REVIEW · #36

Proving Cost Estimation

A proving-cost number from emulation alone. Detailed next.

One standard is open for your review — please comment on PR #36.

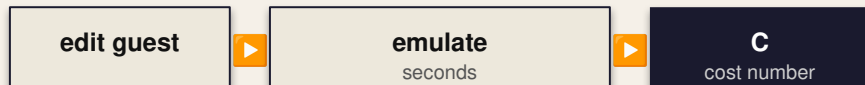
In Review: Proving Cost Estimation

PR #36 · a proving-cost number you get from emulation alone

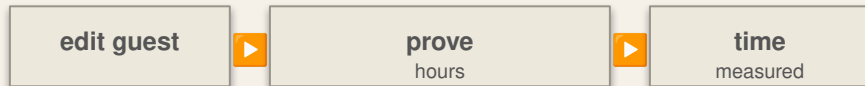
The emulator reports one number, **C**, that tracks how long the prover would take — without producing a proof.

Why it exists

iterate here



not here



What the standard fixes

Linear in proving time

$\text{time} \approx k \cdot C + b$ on a fixed prover and hardware. Push **C** down and proving gets faster — on CPU or GPU alike, because **C** measures work, not time.

Versioned

The cost model is internal, but it carries a version. Numbers are only ever compared within one version.

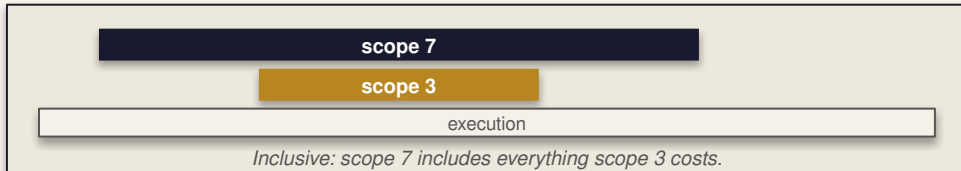
Per-zkVM by design

Each zkVM has its own cost unit. This is an optimization signal — not a benchmark for comparing zkVMs.

Cost attribution scopes — not just how much, but where

```
void zkvm_cost_scope_start(uint8_t scope);  
void zkvm_cost_scope_end(uint8_t scope);
```

A developer brackets a region; the emulator reports what that region cost. The metric is not portable between zkVMs — but the instrumentation is.



Please review PR #36 — [eth-act/zkvm-standards](https://github.com/eth-act/zkvm-standards)

Handover

I am leaving the Ethereum Foundation at the end of August 2026.

Ownership of zkvm-standards passes to

Cody Gunton

mm.bugaj@gmail.com

The remaining slides are the open backlog — four proposals on hold, each for a different reason.

On Hold: Minimum Memory Resources

NEEDS DATA

PR #20 · needs data — the benchmarks have not been run

A floor on how much stack and heap a conforming zkVM must give the guest.

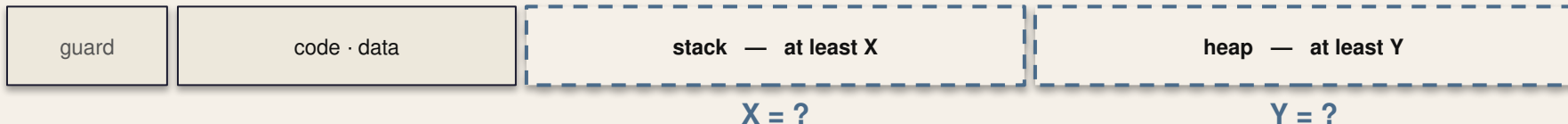
Why it matters

Without a floor, a guest that runs on one zkVM can run out of memory on another. Every other standard assumes that portability — this is the one that would guarantee it.

Why it is stuck

The two numbers in the standard are still blank. Nobody has profiled peak stack and heap for a demanding guest against a large mainnet block.

Guest address space



What is needed before this can be accepted

- Profile peak stack and heap for the most demanding guest programs, against a large mainnet block.
- Document the methodology so the numbers can be reproduced and updated as guests evolve.
- Attach the results to the standard as an appendix.

On Hold: U256 Acceleration

NEEDS DATA

PR #22 · the evidence conflicts — validity not yet established

Should zkVMs expose a standard interface for accelerated 256-bit integer arithmetic?

Two results, three months apart

April 2026 · EF on ZisK

I benchmarked U256 acceleration on ZisK.

Result: no measurable advantage over the software implementation.

Recent · ZisK mulmod opcode

ZisK shipped a mulmod opcode accelerator.

Result: about 40% better proving — on some blocks.

Why that does not settle it

The two are not contradictory. They are under-measured — different baselines, and a sample too small to average.

What a decisive benchmark has to do

- Compare the accelerator against a well-optimized software implementation — not a naive one.
- Run across a few hundred blocks, to learn the amortized improvement rather than the best case.
- Be executed by the Ethereum Foundation, so the methodology is neutral and reproducible.

"Some blocks" is a best case. What matters for Ethereum is the average across real mainnet traffic.

Status

The validity of the U256 Acceleration standard is not yet established.

Closing the PR has been suggested. The Foundation's recommendation: keep it open until the re-run settles the question.

On Hold: Logging Function

NEEDS REVIEW

PR #27 · small, useful — and still waiting for review

A standard function for a guest program to write to the zkVM's logging channel.

- Every zkVM already has logging — each in its own shape. The cost is the fragmentation, not the absence.
- A shared signature is what would let debuggers and trace tooling work across zkVMs.

On Hold: keccak-f[1600] Permutation

NEEDS DECISION

PR #26 · proposed by Rubo, resolves issue #23 — is the finer granularity needed?

We accelerate the whole keccak hash today. This asks for the permutation underneath it.

Today — coarse: one call hashes a finished buffer

`zkvm_keccak256( data, len ) → 32-byte hash`

Proposed — fine: the permutation, so the guest can absorb data as it is produced



Why anyone would want it

Francesco Risitano raised ways to use online hashing in the Ethereum state transition function — absorbing data as it is produced, instead of hashing a completed buffer.

Why it is on hold

It is not yet clear that the finer granularity is actually needed. The validity of the standard has to be reassessed before it moves forward.

Direction: No Modified Toolchains

a recommendation today · a candidate standard tomorrow

Ethereum guest programs should be built with a stock, unmodified toolchain.

Stock toolchain — recommended

Guest application

Standard compiler

trusted · widely tested · unchanged

small, well-understood trusted base

Modified toolchain

Guest application

Standard compiler

Vendor compiler patches ⚠

correctness of ALL emitted code now in question

the Trusted Computing Base grows

The trade-off we accept

A modified compiler could emit code tuned for zkVM execution. We give that up on purpose: a zkVM can absorb the ISA-level cost differences, and correctness is the priority. A smaller Trusted Computing Base is worth more than a faster guest.

Before you go

Review Proving Cost Estimation (#36)

Four proposals need owners: Minimum Memory Resources (#20) U256 Acceleration (#22) Logging Function (#27) keccak-f (#26)

Ownership passes to Cody Gunton at the end of August — please take these up with him.

And if you would like engineering help implementing any standard, the Ethereum Foundation is glad to support.